

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАУЧНО-ОБРАЗОВАТЕЛЬНАЯ КОРПОРАЦИЯ ИТМО»

ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ



Лабораторная работа по предмету
«Системы ввода-вывода» №2

Вариант 4

Выполнил:

Степанов Арсений Алексеевич

Группа:

Р3318 / СВВ 1.1

Преподаватель:

Табунщик Сергей Михайлович

Санкт-Петербург, 2026г.

Цели работы

Написать драйвер символьного устройства, удовлетворяющий требованиям:

- Должен создавать символьное устройство `/dev/varN`, где `N` – это номер варианта
- Должен обрабатывать операции записи и чтения в соответствии с вариантом задания

При записи текста в файл символьного устройства должен осуществляться подсчет введенных цифр. Последовательность полученных результатов (количество цифр) с момента загрузки модуля ядра должна выводиться при чтении файла

Выполнение

Обработчик `var4_write` предназначен для приёма данных из `userspace`, подсчёта количества цифровых символов в переданном буфере и сохранения полученного результата во внутреннем динамическом массиве модуля. Для выполнения этой операции используется вспомогательная функция `count_digits_in_user_buf`, которая поэтапно копирует данные из пользовательской памяти в буфер ядра с помощью `copy_from_user` и подсчитывает количество символов в диапазоне от '0' до '9'. После получения результата функция `var4_write` под защитой мьютекса `results_lock` вызывает функцию `append_result`, добавляющую вычисленное значение в массив результатов. Если текущей ёмкости массива недостаточно, в `append_result` выполняется перевыделение памяти с помощью `krealloc`.

Код функции `var4_write`

```
1 static ssize_t var4_write(struct file *file, const char __user *buf,
2                           size_t count, loff_t *ppos) {
3     u32 digits;
4     int ret;
5
6     ret = count_digits_in_user_buf(buf, count, &digits);
7     if (ret)
8         return ret;
9
10    mutex_lock(&results_lock);
11    ret = append_result(digits);
12    mutex_unlock(&results_lock);
13    if (ret)
14        return ret;
15
16    return count;
17 }
```

Код функции `count_digits_in_user_buf`

```
1 static int count_digits_in_user_buf(const char __user *buf, size_t count,
2                                     u32 *out) {
3     char chunk[256];
4     size_t copied = 0;
5     u32 digits = 0;
6     size_t i;
7
8     while (copied < count) {
9         size_t to_copy = min(sizeof(chunk), count - copied);
10
11         if (copy_from_user(chunk, buf + copied, to_copy))
12             return -EFAULT;
13
14         for (i = 0; i < to_copy; i++) {
15             if (chunk[i] >= '0' && chunk[i] <= '9')
16                 digits++;
17         }
18     }
```

```

18
19     copied += to_copy;
20 }
21
22 *out = digits;
23 return 0;
24 }

```

Код функции `append_result`

```

1 static int append_result(u32 value) {
2     size_t new_capacity;
3     u32 *new_results;
4
5     if (results_count == results_capacity) {
6         new_capacity = results_capacity ? (results_capacity * 2) : 16;
7         new_results =
8             krealloc(results, new_capacity * sizeof(*results), GFP_KERNEL);
9         if (!new_results)
10             return -ENOMEM;
11
12         results = new_results;
13         results_capacity = new_capacity;
14     }
15
16     results[results_count++] = value;
17     return 0;
18 }

```

Обработчик `var4_read` предназначен для передачи пользователю всех ранее накопленных результатов. Сначала под защитой мьютекса создаётся снимок текущего массива результатов с помощью `kmemdup`, что позволяет минимизировать время удержания блокировки. Далее вычисляется необходимый размер текстового представления всех чисел; для этого используется вспомогательная функция `u32_dec_len`, определяющая количество десятичных цифр в каждом значении типа `u32`. После выделения памяти функцией `kmalloс` формируется текстовый буфер, в который результаты записываются построчно с помощью `scnprintf`. Готовые данные копируются в пользовательское пространство через `simple_read_from_buffer`, после чего вся временно выделенная память освобождается функцией `kfree`.

Код функции `var4_read`

```

1 static ssize_t var4_read(struct file *file, char __user *buf, size_t count,
2                          loff_t *ppos) {
3     u32 *snapshot;
4     size_t snapshot_count;
5     size_t i;
6     size_t text_len = 0;
7     char *text;
8     size_t pos = 0;
9     ssize_t ret;
10

```

```

11  mutex_lock(&results_lock);
12  snapshot_count = results_count;
13  if (snapshot_count == 0) {
14      mutex_unlock(&results_lock);
15      return 0;
16  }
17
18  snapshot = kmemdup(results, snapshot_count * sizeof(*results), GFP_KERNEL);
19  mutex_unlock(&results_lock);
20  if (!snapshot)
21      return -ENOMEM;
22
23  for (i = 0; i < snapshot_count; i++)
24      text_len += u32_dec_len(snapshot[i]) + 1;
25
26  text = kmalloc(text_len + 1, GFP_KERNEL);
27  if (!text) {
28      kfree(snapshot);
29      return -ENOMEM;
30  }
31
32  for (i = 0; i < snapshot_count; i++)
33      pos += scnprintf(text + pos, text_len + 1 - pos, "%u\n", snapshot[i]);
34
35  ret = simple_read_from_buffer(buf, count, ppos, text, text_len);
36
37  kfree(text);
38  kfree(snapshot);
39  return ret;
40 }

```

Результаты работы модуля

```

ubuntu@ubuntu:~/lab-2$ dmesg | grep var4
[ 475.643693] var4: loaded, major=239 minor=0
[ 821.222217] var4: unloaded
[ 823.917422] var4: loaded, major=239 minor=0
ubuntu@ubuntu:~/lab-2$

```

```

ubuntu@ubuntu:~/lab-2$ echo "abc" > /dev/var4
ubuntu@ubuntu:~/lab-2$ echo "1a2b3v" > /dev/var4
ubuntu@ubuntu:~/lab-2$ echo "123zzz456" > /dev/var4
ubuntu@ubuntu:~/lab-2$ cat /dev/var4
0
3
6
ubuntu@ubuntu:~/lab-2$

```

Выводы

Я научился работать с модулями ядра Linux и написал драйвер блочного устройства, которое поддерживает операции чтения/записи и обработку полученных данных